# 5. Python: Operators

*"You do not just code. You need to manage how the variables interact"*

**Previous Section:**

</> 4. Python: Conditional Statements

**Contents:**



Operators exist in every programming language. From simple arithmetic like addition and subtraction, to logical comparisons and bitwise operations, operators are what allow programs to perform actions on data.

In Python, operators are special symbols such as `+`, `-`, `*`, `/`, `==`, and `and` that are used to manipulate values and variables. The values involved in these operations are called operands.

Think of operators as the “actions” of programming. Without them, variables would simply store information without ever doing anything useful. Whether we are calculating numbers, comparing conditions, or combining logical statements, operators are what make programs dynamic and interactive.

## 1. Arithmetic Operators

Arithmetic operators are the most fundamental operators in Python. They are used to perform mathematical calculations between values and variables such as addition, subtraction, multiplication, and division.

Think of them as the calculator of programming. Whenever a program needs to compute something—prices, scores, distances, statistics, or even Machine Learning formulas—arithmetic operators are involved.

In Python, arithmetic operators work on operands (values or variables) and return a new result.

**Common Arithmetic Operators:**

| Operator | Meaning | Example |
|---|---|---|
| `+` | Addition | `a + b` |
| `-` | Subtraction | `a - b` |
| `*` | Multiplication | `a * b` |
| `/` | Division | `a / b` |
| `//` | Floor Division | `a // b` |
| `%` | Modulus (Remainder) | `a % b` |
| `**` | Exponentiation (Power) | `a ** b` |

**Example:**

```python
# Input values
a = int(input("Enter value for a: "))
b = int(input("Enter value for b: "))

# Arithmetic Operations
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor Division:", a // b)
print("Modulus:", a % b)
print("Exponentiation:", a ** b)
```

```
Enter value for a: 15
Enter value for b: 4
Addition: 19
Subtraction: 11
Multiplication: 60
Division: 3.75
Floor Division: 3
Modulus: 3
Exponentiation: 50625
```

**Key notes:**

- Normal division `/` always returns a floating-point number.
- Floor division `//` removes the decimal part and keeps only the integer portion.
- The modulus operator `%` returns the remainder after division.
- Exponentiation `**` raises a number to the power of another number.

## 2. Comparison Operators

Comparison operators (also called relational operators) are used to compare two values or variables. Unlike arithmetic operators that return numerical results, comparison operators return Boolean values: `True` , `False` .

These operators are extremely important because programming is built on decision-making. Questions like: Is the user old enough? Is the password correct? Is the score greater than 50? Are two values equal? All rely on comparison operators.

Common Comparison Operators

| Operator | Meaning | Example |
|---|---|---|
| `>` | Greater than | `a > b` |
| `<` | Less than | `a < b` |
| `==` | Equal to | `a == b` |
| `!=` | Not equal to | `a != b` |
| `>=` | Greater than or equal to | `a >= b` |
| `<=` | Less than or equal to | `a <= b` |

Example:

```
# Input values
a = int(input("Enter value for a: "))
b = int(input("Enter value for b: "))

# Comparison Operations
print("a > b:", a > b)
print("a < b:", a < b)
print("a == b:", a == b)
print("a != b:", a != b)
print("a >= b:", a >= b)
print("a <= b:", a <= b)
```

```
Enter value for a: 10
Enter value for b: 7
a > b: True
a < b: False
a == b: False
a != b: True
a >= b: True
a <= b: False
```

Comparison operators always produce either `True` or `False`

- Because 10 is greater than 7 → `a > b` (`10 > 7`) returns `True` → `a > b` (`10 > 7`) returns `False`
- Because 10 is not equal to 7 → `a = b` (`10 = 7`) returns `False` → `a != b` (`10 != 7`) returns `True`

Important Note - Beginners commonly confuse:

| Symbol | Purpose |
|---|---|
| `=` | Assignment |
| `==` | Comparison |

Example: `x = 5` assigns a value; `x == 5` checks if `x` is equal to `5`.

## 3. Boolean Operators

At the core of programming lies a very simple idea. Something is either True or False. Python represents these logical states using Boolean values: `True` and `False` .

Boolean logic is what allows programs to make decisions. Without Boolean values, programs would not know how to make choices, validate conditions, repeat loops, control logic flow. In many ways, Boolean logic is the “thinking system” of programming.

**Boolean Values**

| Boolean | Meaning |
|---|---|
| `True` | Condition is correct |
| `False` | Condition is incorrect |

Example:

```
is_rainy = True
print(is_rainy)
```

True

Boolean values are commonly used inside conditional statements.

```
is_rainy = True

if is_rainy:
    print("Bring an umbrella")
```

Bring an umbrella

Python automatically checks whether `is_rainy` is `True`.

These statements: `if is_rainy:` ; `if is_rainy == True:` ; `if is_rainy is True:` are all the same thing. But the shorter version is considered cleaner and more Pythonic.

## 4. Logical Operators

Python also provides logical operators that combine conditions together.

| Operator | Meaning | Example |
| --- | --- | --- |
| `and` | Both conditions must be True | `a > 0 and b > 0` |
| `or` | At least one condition must be True | `a > 0 or b > 0` |
| `not` | Reverses the Boolean value | `not(a > b)` |

Example:

```python
# Input values
a = int(input("Enter value for a: "))
b = int(input("Enter value for b: "))

# Boolean Operations
print("a > 0 and b > 0:", a > 0 and b > 0)
print("a > 0 or b > 0:", a > 0 or b > 0)
print("not(a > b):", not(a > b))
```

```
Enter value for a: -3
Enter value for b: 4
a > 0 and b > 0: False
a > 0 or b > 0: True
not(a > b): True
```

- For `and`: Both conditions must be true.
  - `print(True and True)` returns `True`
  - `print(True and False)` returns `False`
- For `or` : Only one condition needs to be true.
  - `print(True or False)` returns `True`
  - `print(False or False)` returns `False`
- `not` reverses the result.
  - `print(not True)` returns `False`

Boolean logic becomes extremely important later in Python when working with conditional statements, loops, functions, and other intelligences.

## 4. Bitwise Operators

Most operators we have seen so far work with numbers directly. Bitwise operators are different. Instead of operating on decimal values normally, they work directly on binary digits (bits).

Computers internally store numbers using binary representation (`0` and `1`). Bitwise operators allow us to manipulate those bits directly. These operators are commonly used in networking, cryptography, and image processing.

Although beginners may not use them often at first, understanding bitwise operations helps reveal how computers actually process data internally.

Common Bitwise Operators

| Operator | Meaning | Example |
|---|---|---|
| `&` | Bitwise AND | `a & b` |
| `\|` | Bitwise OR | `a \| b` |
| `~` | Bitwise NOT | `~a` |
| `^` | Bitwise XOR | `a ^ b` |
| `>>` | Right Shift | `a >> 2` |
| `<<` | Left Shift | `a << 2` |

Example:

```
# Input values
a = int(input("Enter value for a: "))
b = int(input("Enter value for b: "))

# Bitwise Operations
print("Bitwise AND:", a & b)
print("Bitwise OR:", a | b)
print("Bitwise NOT of a:", ~a)
print("Bitwise XOR:", a ^ b)
print("Right Shift:", a >> 2)
print("Left Shift:", a << 2)
```

```
Enter value for a: 10
Enter value for b: 4
Bitwise AND: 0
Bitwise OR: 14
Bitwise NOT of a: -11
Bitwise XOR: 14
Right Shift: 2
Left Shift: 40
```

- Bitwise AND `&` returns `1` only if both bits are `1`.

Example:

```
10 = 1010
4  = 0100
-------------
&  = 0000
```

Result: `0`

- Bitwise OR `|` returns `1` if at least one bit is `1`.

```
10 = 1010
4  = 0100
-------------
|  = 1110
```

Result: `14` (the decimal of `1110` )

- Bitwise NOT `~` flips all bits (`1 → 0`, `0 → 1`).

So `~10` becomes `-11`. Since Python uses a signed binary representation, the result becomes negative.

- Bitwise XOR `^` returns `1` only when the bits are different.

```
10 = 1010
4  = 0100
-------------
^  = 1110
```

Result: `14`

- Right Shift `>>` moves bits to the right. So `10 >> 2` is equivalent to dividing by $2^2$. So the result is `2`
- Left Shift `<<` moves bits to the left. So `10 << 2` is equivalent to multiplying by $2^2$. So the result is `40`

## 5. Assignment Operators

Assignment operators are used to assign values to variables. The most basic assignment operator is `=` , which stores the value on the right side into the variable on the left side.

However, Python also provides shorthand assignment operators that combine arithmetic operations with assignment. These operators make code shorter, cleaner, and easier to write.

Common Assignment Operators

| Operator | Meaning | Example |
|---|---|---|
| `=` | Assign value | `a = 5` |
| `+=` | Add and assign | `a += 5` |
| `-=` | Subtract and assign | `a -= 5` |
| `*=` | Multiply and assign | `a *= 5` |
| `/=` | Divide and assign | `a /= 5` |
| `//=` | Floor divide and assign | `a //= 5` |
| `%=` | Modulus and assign | `a %= 5` |
| `**=` | Exponent and assign | `a **= 5` |
| `<<=` | Left shift and assign | `a <<= 2` |
| `>>=` | Right shift and assign | `a >>= 2` |

Example:

```python
# Input value
a = int(input("Enter value for a: "))
# Basic assignment
b = a
print("Assign:", b)
# Add and assign
b += a
print("After += :", b)
# Subtract and assign
b -= a
print("After -= :", b)
# Multiply and assign
b *= a
print("After *= :", b)
# Left shift and assign
b <<= 2
print("After <<= :", b)
```

```
Enter value for a: 12
Assign: 12
After += : 24
After -= : 12
After *= : 144
After <<= : 576
```

- The line `b += a` is equivalent `b = b + a`
- The line `b -= a` is equivalent `b = b - a`
- The line `b *= a` is equivalent `b = b * a`
- The line `b <<= a` is equivalent `b = b << a`

## 6. Identity Operators

Identity operators are used to check whether two variables refer to the exact same object in memory.

This is different from checking if two values are equal. Two variables may contain the same value, but still exist in different memory locations. Python provides two identity operators:

| Operator | Meaning |
| --- | --- |
| `is` | True if both variables are identical |
| `is not` | True if both variables are not identical |

Example:

```
# Input values
a = int(input("Enter value for a: "))
b = int(input("Enter value for b: "))

# c references the same object as a
c = a

print("a is not b:", a is not b)
print("a is c:", a is c)
```

```
Enter value for a: 12
Enter value for b: 14
a is not b: True
a is c: True
```

- `is` checks whether two variables point to the same memory object. So `a is c` results in True.Because `c` directly references `a`.
- `is not` checks whether two variables are different objects. So `a is not b` may result in `True` if they are stored separately.

Difference Between `==` and `is` . One of the most common beginner mistakes in Python is confusing the above two.
Although they may appear similar, they check completely different things.

| Operator | Purpose |
| --- | --- |
| `==` | Checks if values are equal |

| Operator | Purpose |
| --- | --- |
| `is` | Checks if objects are identical in memory |

- Using `==` (Value Equality)

```
x = [1, 2, 3]
y = [1, 2, 3]

if x == y:
    print("x and y have the same values")
else:
    print("x and y do not have the same values")
```

```
x and y have the same values
```

Why? Because both lists contain the same data.

- Using `is` (Memory Identity)

```
x = [1, 2, 3]
y = [1, 2, 3]
z = x

# Case 1
if x is y:
    print("x and y are the same object")
else:
    print("x and y are not the same object")

# Case 2
if x is z:
    print("x and z are the same object")
else:
    print("x and z are not the same object")
```

```
x and y are not the same object
x and z are the same object
```

Understanding the Difference

- `x == y` checks "*Do these variables contain the same value?*"
- `x is y` checks "Are these variables the exact same object in memory?"

## 7. Membership Operators

Membership operators are used to check whether a value exists inside a sequence such as: Lists; Strings; Tuples; Dictionaries; Sets

Python provides two membership operators:

| Operator | Meaning |
|---|---|
| `in` | True if value exists |
| `not in` | True if value does not exist |

Example:

```
# Input values
x = int(input("Enter value for x: "))
y = int(input("Enter value for y: "))

# List
my_list = [10, 20, 30, 40, 50]

# Membership checks
if x not in my_list:
    print("x is NOT present in the list")
else:
    print("x is present in the list")

if y in my_list:
    print("y is present in the list")
else:
    print("y is NOT present in the list")
```

```
Enter value for x: 20
Enter value for y: 24
x is present in the list
y is NOT present in the list
```

- `in` checks whether a value exists inside a sequence. So `20 in my_list` results in `True`
- `not in` checks whether a value does not exist. So `24 not in my_list` results in `True`. Because `24` is not found inside the list.

## 8. Ternary Operator

The ternary operator (also called a conditional expression) is a compact way of writing simple `if-else` statements in a single line.

Instead of writing multiple lines:

```
if condition:
    value = x
else:
    value = y
```

we can simplify it into one line.

```
value_if_true if condition else value_if_false
```

Example:

```
# Input values
a = int(input("Enter value for a: "))
b = int(input("Enter value for b: "))

# Ternary operation
minimum = a if a < b else b

print("Minimum value:", minimum)
```

Enter value for a: 14
Enter value for b: 11
Minimum value: 11

The expression: `a if a < b else b` means: If `a < b` is `True` → return `a`. Otherwise → return `b`

**Equivalent Multi-line Version**

```
if a < b:
    minimum = a
else:
    minimum = b
```

The ternary operator simply makes the code shorter and cleaner for simple conditions.

Another Example:

```
age = int(input("Enter your age: "))

status = "Adult" if age >= 18 else "Minor"

print(status)
```

This is commonly used for value assignments and for a compact code writing.

## 9. Other Operators

As programs become more complex, expressions often contain multiple operators at the same time. This raises an important question:

Which operation should Python perform first? To solve this, Python follows two important concepts: Operator Precedence and Operator Associativity

These rules allow Python to evaluate expressions correctly and consistently.

- **Operator Precedence:** Determines which operator gets evaluated first when multiple operators exist inside the same expression.

Some operators have higher priority than others. For example:

- Multiplication happens before addition

- Exponentiation happens before multiplication
- Comparison happens after arithmetic operations

This works similarly to mathematics.

Example:

```
expr = 10 + 20 * 30
print(expr)
```

610

Why? Because multiplication has higher precedence than addition.

Operator precedence also affects logical conditions:

```
name = input("Enter name: ")
age = int(input("Enter age: "))

if name == "Alex" or name == "John" and age >= 2:
    print("Hello! Welcome.")
else:
    print("Good Bye!!")
```

Enter name: Alex
Enter age: 2
Hello! Welcome.

At first glance, this condition may look confusing. But precedence rules determine the evaluation order. Python evaluates: `name == "John" and age >= 2` first because `and` has higher precedence than `or`. The condition becomes:

```
(name == "Alex") or ((name == "John") and (age >= 2))
```

This is why parentheses are extremely useful for readability.

Common Operator Precedence Table

| Precedence | Operators |
| --- | --- |
| Highest | `()` |
| Exponentiation | `**` |
| Unary Operators | `+x`, `-x`, `~x` |
| Multiplication/Division | `*`, `/`, `//`, `%` |
| Addition/Subtraction | `+`, `-` |
| Bitwise Shift | `<<`, `>>` |
| Bitwise AND | `&` |
| Bitwise XOR | `^` |
| Bitwise OR | ` |
| Comparison | `==`, `!=`, `>`, `<`, `>=`, `<=` |
| Logical NOT | `not` |
| Logical AND | `and` |
| Lowest | `or` |

- Operator Associativity: Sometimes operators have the same precedence level. When this happens, Python uses associativity rules to decide evaluation direction.

Associativity determines whether evaluation happens: Left → Right or Right → Left. Most operators in Python use left-to-right associativity.

Example: Left-to-Right Associativity

```
print(100 / 10 * 10)
```

100.0

Both `/` and `*` have the same precedence. So Python evaluates from left to right.

Another Example

```
print(5 - 2 + 3)
```

```
6
```

Parentheses Override Associativity

```
print(5 - (2 + 3))
```

```
0
```

Parentheses always have the highest precedence.

Right-to-Left Associativity

Exponentiation (`**`) uses right-to-left associativity.

```
print(2 ** 3 ** 2)
```

```
512
```

## Summary

This section explored some of the most important concepts behind how Python evaluates expressions and compares objects.

We began with operator precedence, where Python follows a priority system to decide which operations should execute first. Just like mathematics, multiplication occurs before addition, logical operators follow their own hierarchy, and parentheses can completely change evaluation order.

After that, we explored operator associativity, which determines evaluation direction when operators share the same precedence. Most operators evaluate from left to right, while exponentiation evaluates from right to left.

Finally, we examined one of the most misunderstood topics for beginners: the difference between `==` and `is`.

- `==` checks whether two variables contain the same value.
- `is` checks whether two variables reference the exact same object in memory.

This distinction becomes extremely important later when working with objects, lists, functions, classes, and larger Python applications.

By understanding precedence, associativity, and identity comparison, you are no longer just writing Python code—you are beginning to understand how Python thinks internally when evaluating expressions and managing objects in memory.

## References

https://www.geeksforgeeks.org/python/python-operators/

https://www.geeksforgeeks.org/python/difference-between-and-is-operator-in-python/

### Next Section:

</> 6. Python: Built-in Methods